

88. Une méthode : ce qu'elle coûte, ce qui reste refusé

``troupe[i].avancer()`` plutôt que ``avancer(i)``.

1. Ce qu'il faut comprendre

Une méthode s'écrit dans la `struct`, et travaille sur l'objet devant le point. Dans son corps, les champs s'écrivent sous leur nom nu : `x++`, et non `this->x++`.

Ce qu'elle coûte, exactement : l'adresse de l'objet est rangée dans **deux octets réservés à cette méthode**, puis c'est le même `call` qu'ailleurs. Pas de table de fonctions, pas de pile d'objets — et le compilateur compte ces deux octets comme les autres.

Ce qui reste refusé, et pourquoi : `class` (le message propose `struct`), `virtual` (il faudrait une table de fonctions et un appel indirect), l'héritage (il déplacerait les décalages des champs), les constructeurs (du code implicite à la déclaration, alors que les globales prennent leur valeur avant `main`). Et une méthode **ne prend pas d'argument** : elle reçoit déjà son objet.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **630 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          49 o ← la console, à l'allumage - la mise en route de la console
Ennemi::avancer()   16 o ← main() - ta fonction
Ennemi::vivant()    21 o ← main() - ta fonction
main()              232 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
AttendreAcces       12 o ← EcrireNombre - attend de pouvoir écrire une case
AdresseCase         10 o ← nombre() - trouve l'adresse d'une case de l'écran
EcrireNombre        33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles        31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage - vide les deux cartes de l'écran
Tuiles              160 o ← CopierTuiles - 10 lettres sur 43

Lettres gravées : 0123456789
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
nombre() × 1      46 o sur place, dans main() + AdresseCase (10 o), EcrireNombre (33 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : trois ennemis ; seuls les VIVANTS avancent. Leurs x s'affichent :  
// le premier et le troisième montent, celui du milieu (mort) reste à 60.  
//  
// Une MÉTHODE est une fonction rangée DANS la struct.  
// Dedans, x et vie sont ceux de l'ennemi sur qui on l'appelle :  
// troupe[2].avancer(); → c'est le x de troupe[2] qui augmente.  
  
#include <nombre> // écrit un nombre en chiffres
```

2. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {  
    uint8_t x, y, vie;  
  
    void avancer() { x++; } // Méthode : un pas à droite.  
    uint8_t vivant() { return vie > 0; } // Méthode : rend 1 s'il a encore de la vie.  
};
```

3. UN TABLEAU — TROUPE

```
Ennemi troupe[3]; // 3 ennemis.
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

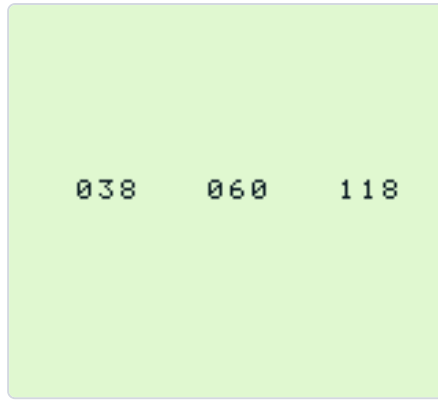
```
int main() { // Le jeu commence ici.  
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }  
    // x = 20, 60, 100 ; 3 vies chacun.  
    troupe[1].vie = 0; // Celui du milieu est mort.  
  
    while (true) { // Répète pour toujours :  
        for (uint8_t i = 0; i < 3; i++) // pour chaque ennemi :  
            if (troupe[i].vivant()) troupe[i].avancer(); // s'il est vivant, il avance.  
  
        for (uint8_t i = 0; i < 3; i++) nombre(3 + i * 6, 8, troupe[i].x); // Affiche les 3 x.  
        image(); // Attend l'image suivante.  
    }  
}
```

3. Ce qui se passe, étape par étape

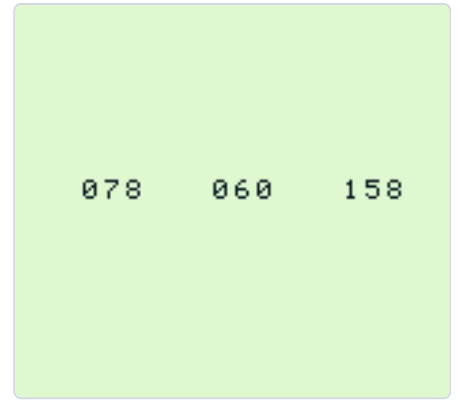
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



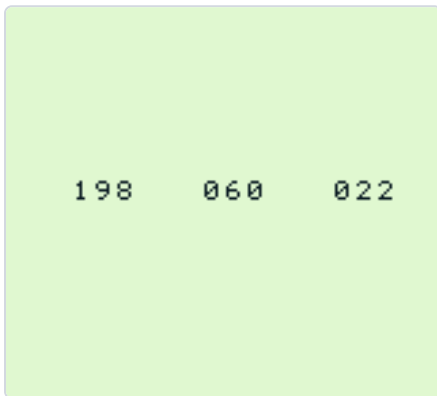
la toute première image — la console vient de démarrer — image 1



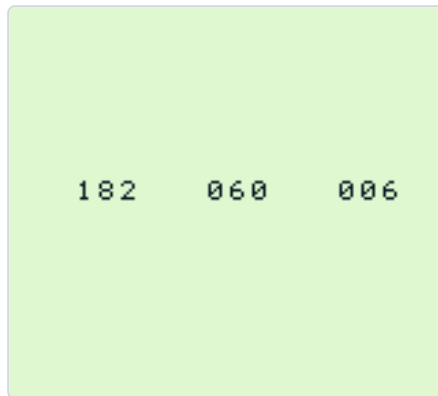
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Deux nombres qui montent, et celui du milieu immobile.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le premier avance (038 → 078)
- ✓ le MORT du milieu ne bouge pas (060)
- ✓ le troisième avance (118 → 158)
- ✓ la méthode a donc écrit dans le BON objet

Ensuite : **89. Les trois façons de faire la même chose** — chapitre 13 : Ce qui va ensemble. · [gameboy3](#) — écrire une cartouche Game Boy en C++